# NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis

Ben Mildenhall[1] Pratul P. Srinivasan[1] Matthew Tancik[1]
Jonathan T. Barron[2] Ravi Ramamoorthi[3] Ren Ng[1]

[1]UC Berkeley

[2]Google Research [3]UC San Diego

ECCV, 2020

Western UNIVERSITY · CANADA

# Table of Contents


1. NeRF
2. Rendering with Radiance Field
3. Training NeRF

# Table of Contents

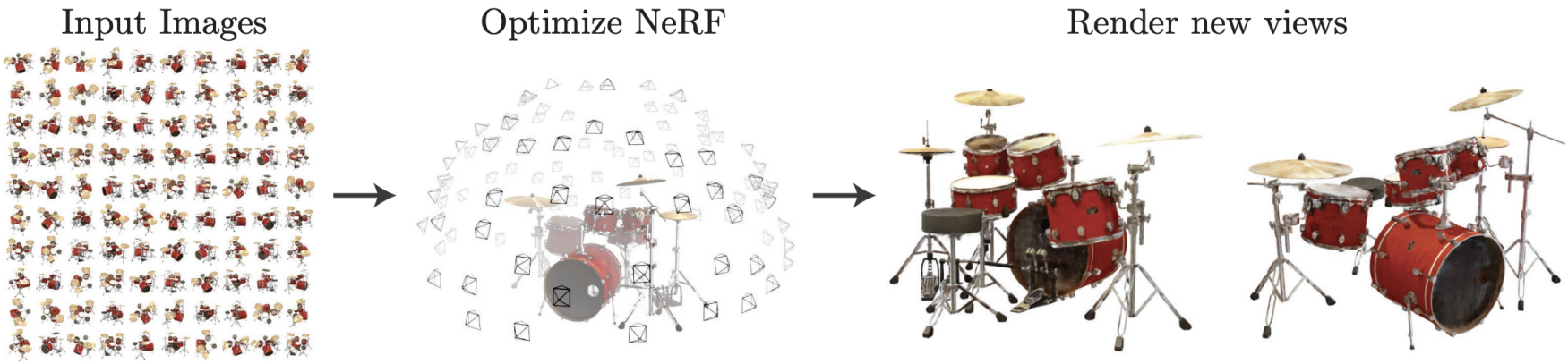
1 NeRF

2 Rendering with Radiance Field

3 Training NeRF

# The Objective of NeRF

- Given a set of input images, optimize a continuous volumetric representation of a scene.
- fit a multilayer perceptron (fully connected network) to implicitly represent just one 3D scene



# What NeRF Represents

- NeRF models a **continuous 5D radiance field**:
  - **3D position:** $\mathbf{x} = (x, y, z)$
  - **Viewing direction:** $\mathbf{d} = (\phi, \theta)$
- It outputs:
  - **Volume density** $\sigma(\mathbf{x}) \geq 0$
  - **View-dependent color** $\mathbf{c}(\mathbf{x}, \mathbf{d}) \in [0, 1]^3$
- So the neural field is:

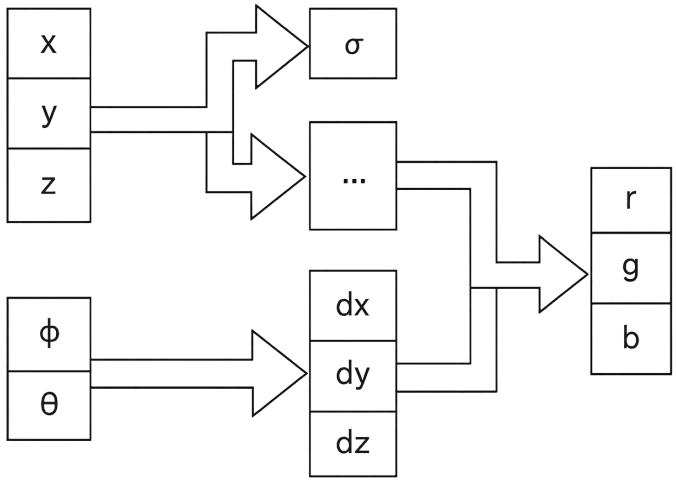
$$F_\theta : (\mathbf{x}, \mathbf{d}) \mapsto (\sigma, \mathbf{c})$$



# NeRF as Branched MLP

- In practice, it is implemented as an MLP with a "branch":
  - density $\sigma$ depends on $\mathbf{x}$
  - color $\mathbf{c}$ depends on $\mathbf{x}$ and $\mathbf{d}$

- **Question:** Why density depends only on 3D location and color depends on location and viewing direction?



# Answer

- Volume density predicted as a function of location
  - Encourages multiview consistency
  - Only a few materials have density/opacity that depends on viewing direction: polaroid glass etc.
- RGB color predicted as a function of location and direction
  - Captures light transport properties

# NeRF Model

- Multilayer Perceptron (MLP)
  - Fully connected, feed-forward network
- 3D coordinate processed through 8 layers with ReLU activation
  - Outputs density and a 256-dimensional feature vector
- Feature vector and viewing direction processed through a single layer
  - Outputs RGB color value

Western UNIVERSITY · CANADA

# Table of Contents


1. NeRF
2. Rendering with Radiance Field
3. Training NeRF

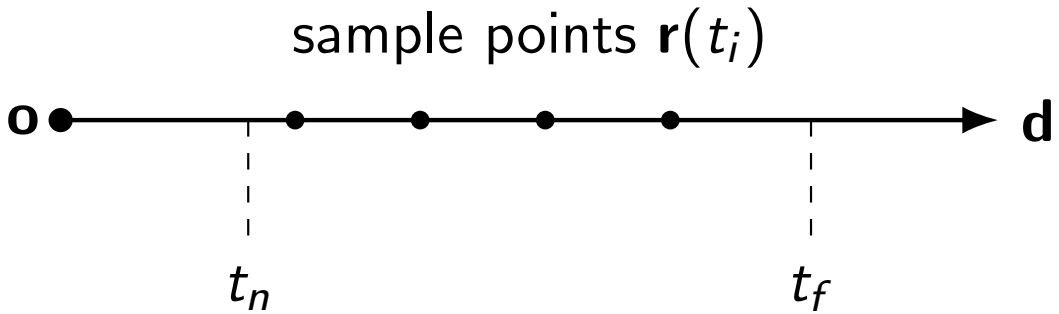
# Rendering a Pixel: A Ray Through Radiance Field

A camera pixel corresponds to a **ray**:

$$\mathbf{r}(t) = \mathbf{o} + t\,\mathbf{d}, \qquad t \in [t_n, t_f]$$

- $\mathbf{o}$: camera origin
- $\mathbf{d}$: ray direction
- $[t_n, t_f]$: near / far bounds along the ray

**Ray Parameterization**



# The Volumetric Rendering Equation

NeRF uses standard **emission–absorption volume rendering**. The color of a ray is:

$$\mathbf{C}(\mathbf{r}) = \int_{t_n}^{t_f} T(t)\,\sigma(\mathbf{r}(t))\,\mathbf{c}(\mathbf{r}(t), \mathbf{d})\,dt$$

where $T(t)$ is the **transmittance** (probability the ray has not been absorbed before $t$):

$$T(t) = \exp\left(-\int_{t_n}^{t} \sigma(\mathbf{r}(s))\,ds\right)$$

**Interpretation:**

- $\sigma(\cdot)$ is "how likely you hit matter"
- $\mathbf{c}(\cdot)$ is the color emitted at that point (depends on view direction)
- $T(t)$ downweights points that are "behind" dense stuff

# Discretizing Volume Rendering (1/2)

NeRF approximates the volume rendering integral by sampling $N$ points along a ray:

$$t_1, \ldots, t_N, \qquad \Delta_i = t_{i+1} - t_i.$$

Evaluate the MLP at $\mathbf{x}_i = \mathbf{r}(t_i)$:

$$(\sigma_i, \mathbf{c}_i) = F_\theta(\mathbf{x}_i, \mathbf{d}).$$

Define the per-sample alpha (opacity) using a constant-density segment approximation:

$$\alpha_i = 1 - \exp(-\sigma_i \, \Delta_i) \, .$$

Define accumulated transmittance up to sample $i$:

$$T_i = \prod_{j=1}^{i-1} (1 - \alpha_j), \qquad T_1 = 1.$$

# Discretizing Volume Rendering (2/2)

The rendered color is a weighted sum:

$$\hat{\mathbf{C}}(\mathbf{r}) = \sum_{i=1}^{N} w_i \, \mathbf{c}_i, \qquad w_i = T_i \, \alpha_i.$$

Often you also render (expected) depth:

$$\hat{D}(\mathbf{r}) = \sum_{i=1}^{N} w_i \, t_i.$$

**Weights along the ray**

| | | | | | |
|---|---|---|---|---|---|
| **ray samples:** | $\mathbf{x}_1$ | $\mathbf{x}_2$ | $\mathbf{x}_3$ | $\cdots$ | $\mathbf{x}_N$ |
| | $\vert$ | $\vert$ | $\vert$ | | $\vert$ |
| **density** $\sigma$**:** | $\sigma_1$ | $\sigma_2$ | $\sigma_3$ | $\cdots$ | $\sigma_N$ |
| **alpha** $\alpha$**:** | $\alpha_1$ | $\alpha_2$ | $\alpha_3$ | $\cdots$ | $\alpha_N$ |
| **weights** $w$**:** | $w_1$ | $w_2$ | $w_3$ | $\cdots$ | $w_N$ |

($\sum_{i=1}^{N} w_i \approx 1$ if far bound covers scene)

# Table of Contents


1 NeRF

2 Rendering with Radiance Field

3 Training NeRF

# NeRF Training Pipeline (Overview)

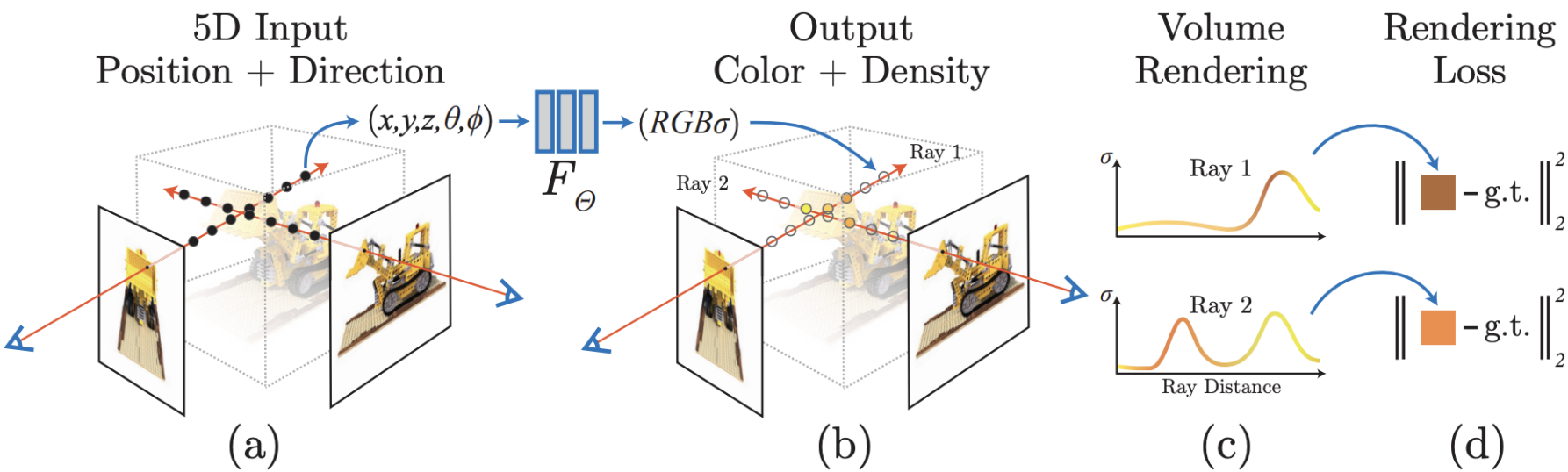



Figure 1: Overview: 5D input $(\mathbf{x}, \mathbf{d}) \rightarrow (\mathbf{c}, \sigma)$, volume rendering along rays, and photometric loss.

# Training Objective (Photometric Loss)

- Given posed images (known camera intrinsics / extrinsics), we sample many rays and minimize the mean squared error (MSE) between rendered and ground-truth pixel colors:

$$\mathcal{L}(\theta) = \sum_{\mathbf{r}\in\mathcal{R}} \left\| \hat{\mathbf{C}}(\mathbf{r}) - \mathbf{C}^*(\mathbf{r}) \right\|_2^2$$

- NeRF uses **two networks** (coarse and fine). So the total loss is:

$$\mathcal{L} = \sum_{\mathbf{r}\in\mathcal{R}} \left( \left\| \hat{\mathbf{C}}_c(\mathbf{r}) - \mathbf{C}^*(\mathbf{r}) \right\|_2^2 + \left\| \hat{\mathbf{C}}_f(\mathbf{r}) - \mathbf{C}^*(\mathbf{r}) \right\|_2^2 \right)$$

Western UNIVERSITY · CANADA

# References




Raleigh Hansen, Rory Little, Dylan Thompson,

*CSCI 480/580 - Computer Graphics*,

Western Washington University, 2022.



B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng,

*NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis*,

ECCV, 2020.



S. M. Seitz and C. R. Dyer,

*Photorealistic Scene Reconstruction by Voxel Coloring*,

IJCV, 1999.